Université Libre de Bruxelles

INFO-H503

GPU computing

CUDA Benchmark Mini-Project

Antoine Berthion 566199
Louis Halkin 500859

March 17, 2026

Supervised by:
Bonatto Daniele and Soetens Eline

Table des matières

1 Introduction **3**

2 Configuration Matérielle **3**

3 Méthodologie **3**

4 Analyse des Résultats **4**
4.1 Impact du travail par fil d'exécution (j) 4
4.2 Modèle Roofline et intensité arithmétique 5

5 Utilisation des LLMs **5**

6 Conclusion **6**

A Annexe: Kernels CUDA **7**

1 Introduction

L'objectif de ce projet est d'évaluer l'efficacité ainsi que les limitations architecturales d'un GPU **RTX 4070** en développant et en profilant deux opérations arithmétiques fondamentales. En **calcul haute performance**, la compréhension du lien entre les types de données, les opérations effectuées et l'architecture matérielle est essentielle afin d'optimiser les performances.

Ce projet se concentre sur trois dimensions principales de la performance :

1. **L'échelle des données :** analyser l'évolution des performances en fonction de la taille des **tableaux**.
2. **La capacité des fils d'exécution (threads) :** étudier l'impact de l'assignation de plusieurs opérations à chaque **fil d'exécution** sur l'**occupation** du GPU et sur le masquage de la latence mémoire.
3. **L'intensité arithmétique :** observer la transition entre un régime **limité par la mémoire** (*memory-bound*) et un régime **limité par le calcul** (*compute-bound*).

Afin de profiler la performance, des **noyaux de calcul (kernels)** réalisant des opérations d'addition et de multiplication sur des nombres à virgule flottante (*floats*) ont été implémentés. Les expériences ont été automatisées afin de faire varier systématiquement les trois paramètres décrits précédemment.

Une analyse basée sur le **modèle Roofline** a également été réalisée. Ce modèle fournit une représentation visuelle reliant les performances en GFLOPS à l'intensité arithmétique. Il permet d'identifier si les performances observées sont limitées par les **noyaux de calcul** eux-mêmes ou plutôt par la **bande passante mémoire** disponible sur le GPU, ou encore par le plafond théorique de performance de l'architecture.

2 Configuration Matérielle

Les expérimentations ont été réalisées sur un GPU grand public de génération récente [1]. Les spécifications matérielles identifiées sont les suivantes :

- **Modèle :** NVIDIA GeForce RTX 4070
- **Mémoire VRAM :** 12010 MB (12 GB).
- **Version du pilote :** 32.0.15.9174.

3 Méthodologie

Afin d'évaluer les performances des opérations étudiées, une série d'expériences contrôlées a été mise en place à l'aide de kernels CUDA simples (Voir annexe A). Deux noyaux ont été implémentés : l'un réalisant une addition élémentaire et l'autre une multiplication sur des tableaux de nombres à virgule flottante.

Les expériences consistent à exécuter ces noyaux sur des tableaux de différentes tailles N. Chaque thread traite un certain nombre d'éléments consécutifs, contrôlé par un paramètre J, permettant d'étudier l'impact du travail par thread sur l'utilisation des ressources du GPU. Nous disposons également d'un paramètre K permettant de contrôler la charge sur chaque thread, en répétant les opérations.

Afin d'améliorer la précision des mesures et de réduire l'influence des fluctuations temporelles, chaque configuration expérimentale est exécutée plusieurs fois, et le temps d'exécution moyen est calculé à partir de ces répétitions. Cette approche permet d'obtenir une estimation plus stable des performances.

[1] Les expérimentations ont également été menées sur un GPU GTX 1060. Nous n'avons pas inclu ces derniers résultats.

Les métriques principales observées sont :

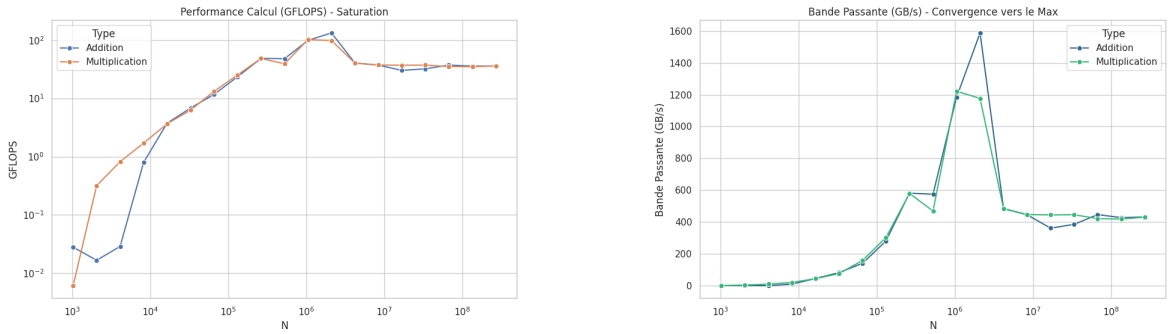
- le temps d'exécution moyen des noyaux,
- les performances en GFLOPS,
- la bande passante mémoire effective.

Les performances en GFLOPS sont calculées à partir du nombre total d'opérations effectuées par les noyaux et du temps d'exécution moyen. De manière similaire, la bande passante mémoire effective est estimée en fonction du volume de données transférées entre la mémoire globale et les unités de calcul.

4 Analyse des Résultats

Les expériences initiales, présentées dans les figures `gflops.png` et `memory.png`, montrent que les performances en GFLOPS ainsi que la bande passante mémoire utilisée augmentent conjointement avec la taille du tableau N. Une saturation progressive est observée pour $N > 10^6$, indiquant que le GPU atteint alors son régime de fonctionnement maximal.

La bande passante atteint approximativement 1600 GB/s, ce qui suggère une utilisation efficace de la hiérarchie mémoire du GPU, notamment des mécanismes de cache, avant de se stabiliser au niveau du débit global de la mémoire.



(a) Performance (GFLOPS)

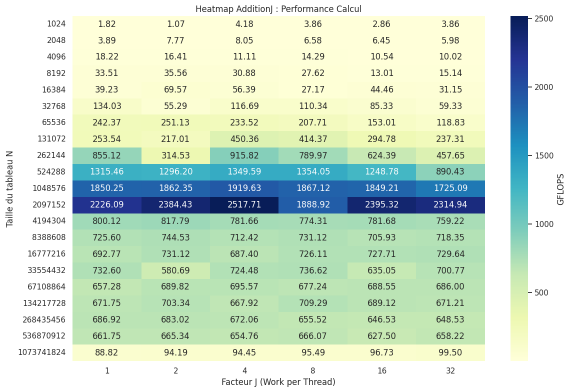
(b) Bande passante (GB/s)

Figure 1: Évolution des performances et de la bande passante mémoire en fonction de la taille N du tableau.

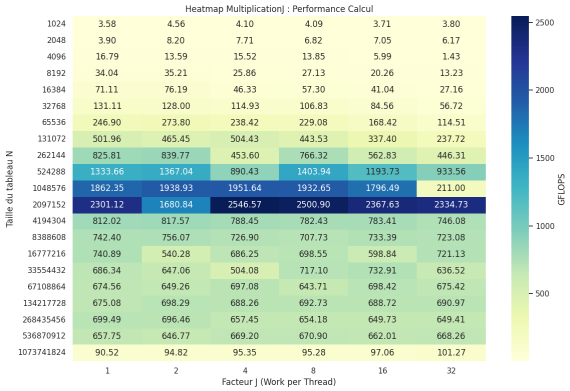
4.1 Impact du travail par fil d'exécution (j)

Les cartes thermiques (*heatmaps*) permettent d'analyser l'influence du facteur J, correspondant au nombre d'éléments traités par chaque thread. Les résultats indiquent que, pour des tailles de tableaux intermédiaires, l'augmentation de J permet d'améliorer les performances.

Ce phénomène s'explique par une meilleure utilisation des registres et par une réduction relative du coût des accès mémoire globaux et des opérations d'indexation. Cette optimisation se manifeste visuellement dans les zones bleu foncé des figures, correspondant aux performances les plus élevées.



(a) Performance : Addition



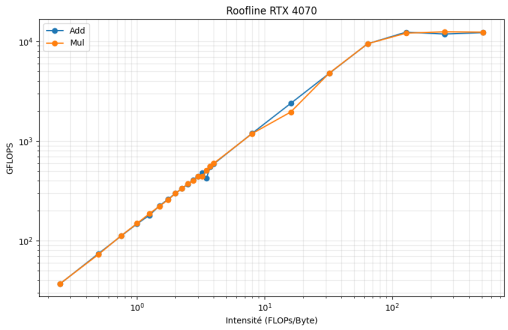
(b) Performance : Multiplication

Figure 2: Performance en GFLOPS en fonction de la taille du tableau N et du facteur de travail J.

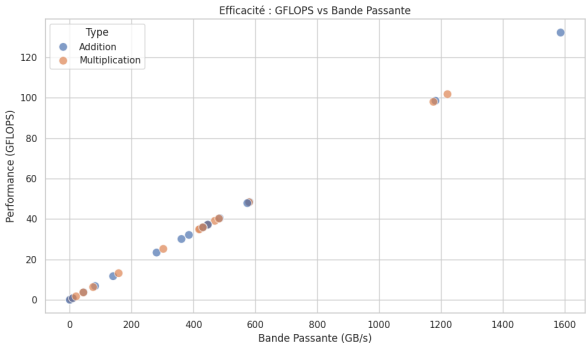
4.2 Modèle Roofline et intensité arithmétique

Le graphique *roofline.png* constitue un élément central de l'analyse. Il montre que les noyaux de calcul implémentant les opérations d'addition et de multiplication suivent une tendance linéaire ascendante, caractéristique d'un régime **limité par la mémoire** (*memory-bound*). Cela indique que les performances sont principalement contraintes par la bande passante mémoire plutôt que par la capacité de calcul du GPU.

L'augmentation de l'intensité arithmétique (k) permet progressivement de se rapprocher du plafond théorique de performance du GPU. Cette observation est également confirmée par la corrélation visible entre les GFLOPS et la bande passante mémoire dans le graphique *scatter_performance.png*.



(a) Modèle Roofline



(b) GFLOPS vs Bande passante

Figure 3: Analyse de l'intensité arithmétique et de l'efficacité des noyaux de calcul.

5 Utilisation des LLMs

Dans cette courte partie, nous discuterons de l'utilisation des LLM dans le cadre du projet. Aucun LLM n'a été utilisé pour l'aspect implémentation ainsi que la compréhension du projet. Cependant, ce rapport à été corrigé du point de vue de la syntaxe et de la grammaire par DeepL ainsi qu'un modèle de langage. Il est à noter qu'aucune des informations du rapport n'a été produite par une autre personne que les auteurs.

6 Conclusion

Ce projet met en évidence que, sur une architecture GPU moderne telle que la RTX 4070, les opérations arithmétiques simples sont fortement dépendantes de la vitesse de transfert des données en mémoire. En conséquence, l'exploitation optimale du potentiel de calcul (TFLOPS) nécessite une augmentation de l'intensité arithmétique k, permettant de déplacer l'exécution vers un régime **limité par le calcul** (*compute-bound*).

A Annexe: Kernels CUDA

Dans cette annexe, nous présentons les principaux noyaux CUDA utilisés pour les expérimentations.

vecAddJ Kernel

```
__global__ void vecAddJ(const float* a, const float* b, float* c, int n, int
    j, int k) {

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    // each thread processes j elements, so we loop over j and compute the
        corresponding index for each element
    for (int t = 0; t < j; t++) {
        int i = idx + t * stride;
        if (i < n) {
            float v1 = a[i];
            float v2 = b[i];
            // perform k additions to increase the computational load
            for (int l = 0; l < k; l++) {
                v1 += v2;
            }
            c[i] = v1;
        }
    }
}
```

vecMulJ Kernel

```
__global__ void vecMulJ(const float* a, const float* b, float* c, int n, int
    j, int k) {

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    // each thread processes j elements
    for (int t = 0; t < j; t++) {
        int i = idx + t * stride;
        if (i < n) {
            float v1 = a[i];
            float v2 = b[i];

            // perform k multiplications to increase the computational load
            for (int l = 0; l < k; l++) {
                v1 *= v2;
            }

            c[i] = v1;
        }
    }
}
```